

Universidade Católica de Brasília

Campus Taguatinga | Engenharia de Software | Teste de Software

Sistema Archivum

Documento de Visão e Histórias de Usuário

Versão 2.0

Equipe de Desenvolvimento

Alexandre Rodrigues Costa
Caio Murilo Silva de Oliveira
Caian Dieter Passos
Ester Yasmin Santos Silva
André Luiz Borges Aguiar
Artur Braz Lopes

Brasília, DF — Maio de 2026

Histórico de Revisões

Data	Versão	Descrição	Autores
21/05/2026	1.0	Criação inicial do Documento de Visão.	Toda a equipe
27/05/2026	2.0	Revisão completa: escopo, stakeholders, requisitos, restrições e cenários BDD expandidos.	Toda a equipe

Sumário

Histórico de Revisões..... 2

Sumário 3

1. Visão do Produto

O Archivum é uma aplicação web voltada para a área da Educação, desenvolvida em Python com o framework Flask. Seu propósito central é permitir que estudantes do ensino superior registrem, organizem e gerenciem fichamentos e o acervo das obras literárias com as quais trabalham ao longo da vida acadêmica, de forma simples, intuitiva e acessível a partir de qualquer navegador moderno.

O sistema foi concebido para eliminar a dependência de planilhas e anotações dispersas, oferecendo uma interface centralizada onde o estudante realiza login seguro, cadastra obras informando título e autor, edita registros existentes e os remove quando necessário. Toda a informação é persistida em banco de dados SQLite (archivum.db), garantindo durabilidade dos dados entre sessões.

Tecnicamente, o projeto segue uma arquitetura modular com separação clara entre camadas: a lógica de rotas (routes.py), o acesso a dados (database.py) e a interface do usuário (HTML, CSS e JavaScript Vanilla). Essa separação facilita a manutenção, os testes automatizados e a evolução incremental do sistema.

2. Escopo do Sistema

2.1 O que está no escopo

- Autenticação de usuário via login e senha com retorno de token de sessão.
- Cadastro de obras literárias (título e autor) — operação Create do CRUD.
- Listagem de todas as obras cadastradas pelo usuário autenticado — operação Read.
- Edição de título e/ou autor de uma obra existente — operação Update.
- Exclusão de uma obra do acervo — operação Delete.
- Persistência de dados em banco SQLite entre sessões do usuário.
- Interface web responsiva acessível via navegador (localhost:3000).
- Suíte de testes automatizados: unitários/API com Pytest e End-to-End com Playwright.

2.2 O que está fora do escopo (nesta versão)

- Cadastro de novos usuários diretamente pela interface (o usuário padrão é configurado no banco).
- Recuperação de senha ou autenticação via provedores externos (OAuth, SSO).
- Testes de carga, estresse, penetração e segurança avançada.
- Integração com sistemas externos (APIs de bibliotecas, exportação para PDF/CSV etc.).
- Funcionalidade de fichamento textual completo (resumos, citações, notas) — escopo futuro.
- Deploy em ambiente de produção em nuvem (sistema desenvolvido para uso local).

3. Stakeholders

Os principais envolvidos e interessados no sistema Archivum são:

Stakeholder	Tipo	Interesse / Responsabilidade
Equipe de Desenvolvimento	Interno	Projetar, implementar, testar e documentar o sistema. Responsável por todas as entregas.
Estudantes Universitários	Usuário Final	Utilizar o sistema para registrar e organizar obras literárias de seus cursos.
Professor / Avaliador	Externo / Avaliador	Avaliar a qualidade do sistema, dos testes e da documentação produzida pela equipe.
UCB — Instituição de Ensino	Patrocinador Acadêmico	Prover infraestrutura educacional e contexto para o desenvolvimento do projeto.

4. Requisitos Funcionais

ID	Descrição	Prioridade	US Vinculada
RF01	O sistema deve exigir autenticação (usuário e senha) para liberar acesso às funcionalidades.	Alta	US01
RF02	O sistema deve validar as credenciais informadas contra os dados armazenados no banco SQLite.	Alta	US01
RF03	O sistema deve retornar token de autenticação após login bem-sucedido (HTTP 200).	Alta	US01
RF04	O sistema deve bloquear acesso e retornar HTTP 401 para credenciais inválidas.	Alta	US01
RF05	O sistema deve permitir ao usuário autenticado cadastrar uma nova obra informando título e autor.	Alta	US02
RF06	O sistema deve retornar HTTP 201 e o ID gerado ao criar uma obra com sucesso.	Alta	US02
RF07	O sistema deve listar todas as obras cadastradas do acervo via GET /api/livros (HTTP 200).	Alta	US03
RF08	O sistema deve exibir título, autor e botões de ação (Editar, Excluir) para cada obra na lista.	Média	US03
RF09	O sistema deve permitir ao usuário editar título e/ou autor de uma obra existente via PUT.	Alta	US04
RF10	O sistema deve manter o mesmo ID da obra durante o fluxo de edição.	Alta	US04
RF11	O sistema deve permitir ao usuário excluir uma obra do acervo via DELETE.	Alta	US05
RF12	O sistema deve confirmar a exclusão retornando HTTP 200 e mensagem de sucesso.	Alta	US05

5. Requisitos Não Funcionais

ID	Categoria	Descrição
----	-----------	-----------

RNF01	Desempenho	Os 3 testes automatizados da suíte Pytest devem ser executados em menos de 2 segundos em ambiente local.
RNF02	Usabilidade	A interface deve ser operável sem treinamento por estudantes com conhecimento básico de navegadores web.
RNF03	Confiabilidade	A taxa de sucesso dos casos de teste automatizados deve ser de 100% ao final do ciclo de desenvolvimento.
RNF04	Manutenibilidade	O código deve ser organizado em módulos separados (routes.py, database.py) para facilitar manutenção e testes.
RNF05	Portabilidade	O sistema deve funcionar nos sistemas operacionais Windows, Linux e macOS com Python 3.10+.
RNF06	Testabilidade	O banco de dados de testes deve ser criado em memória (SQLite in-memory) para isolamento total dos dados de produção.
RNF07	Compatibilidade	A interface frontend deve ser compatível com os navegadores Google Chrome, Mozilla Firefox e Microsoft Edge nas versões atuais.

6. Restrições do Projeto

Tecnologia	Backend obrigatoriamente em Python (Flask). Frontend em HTML, CSS e JavaScript Vanilla — sem frameworks como React ou Vue.
Banco de Dados	Uso exclusivo de SQLite (archivum.db) para persistência. Não são permitidos bancos relacionais externos (MySQL, PostgreSQL) nesta versão.
Autenticação	O mecanismo de autenticação é simplificado (token fixo "autenticado"). Não é permitido o uso de JWT ou OAuth neste escopo acadêmico.
Ferramentas	Testes automatizados devem ser implementados exclusivamente com Pytest (API/unitários) e Playwright (E2E). Outras ferramentas de teste estão fora do escopo.
Ambiente	O sistema é desenvolvido e executado exclusivamente em ambiente local (localhost:3000). Não há deploy em nuvem ou servidores externos.
Prazo	O projeto segue o calendário da disciplina Teste de Software da UCB Taguatinga, com entrega até o final do semestre letivo de 2026/1.
Equipe	O desenvolvimento é realizado por seis estudantes; não há orçamento financeiro disponível. Todas as ferramentas utilizadas são gratuitas ou open-source.

7. Visão da Arquitetura

O Archivum adota uma arquitetura em três camadas, com responsabilidades claramente separadas para favorecer a testabilidade e a manutenibilidade do código:

Camada	Artefato	Responsabilidade
Apresentação (Frontend)	<code>index.html</code> , <code>style.css</code> , <code>script.js</code>	Renderiza a interface do usuário, captura inputs e realiza chamadas <code>fetch()</code> para a API REST.
Aplicação (Backend)	<code>app.py</code> , <code>routes.py</code>	Inicializa o servidor Flask, define rotas HTTP e orquestra a lógica de negócio.
Dados (Persistência)	<code>database.py</code> , <code>archivum.db</code>	Gerencia a conexão com o SQLite, executa queries e retorna resultados às rotas.
Testes	<code>tests/test_api.py</code> , <code>tests/test_e2e.py</code>	Testes unitários/API com Pytest e testes End-to-End com Playwright.

8. Histórias de Usuário e Cenários BDD

As histórias de usuário estão descritas no formato padrão ("Como... quero... para...") e seus critérios de aceitação são detalhados em cenários BDD (Behavior-Driven Development) com a linguagem Gherkin (Dado / Quando / Então). Cada história mapeia diretamente para um caso de uso e para os requisitos funcionais definidos na Seção 4.

US01 — Autenticação de Usuário	
História	Como estudante, quero autenticar meu login no sistema, para que apenas eu possa acessar e gerenciar meu acervo pessoal de obras.
Critérios de Aceitação	- O sistema deve exibir campos de Usuário e Senha na tela inicial. - Ao enviar credenciais corretas, o sistema deve retornar HTTP 200 e token de sessão. - Ao enviar credenciais incorretas, o sistema deve retornar HTTP 401 e mensagem de erro. - Campos em branco não devem ser submetidos à API; o sistema exibe validação client-side.
Cenários BDD	Cenário 1: Login com sucesso Dado que o usuário acessa localhost:3000 Quando informa usuário 'admin' e senha '123' e clica em Entrar Então o sistema retorna HTTP 200 com {"token": "autenticado"} E redireciona para a tela de Acervo de Livros Cenário 2: Login com senha incorreta Dado que o usuário acessa a tela de login Quando informa usuário 'admin' e senha '999' e clica em Entrar Então o sistema retorna HTTP 401 E exibe mensagem 'Usuário ou senha inválidos' Cenário 3: Login com campos em branco Dado que o usuário acessa a tela de login Quando deixa o campo Senha em branco e clica em Entrar Então o sistema exibe validação sem enviar requisição à API

US02 — Adicionar Obra ao Acervo

História	Como estudante autenticado, quero adicionar uma nova obra literária informando título e autor, para que eu possa manter meu acervo acadêmico organizado no sistema.
Critérios de Aceitação	- O sistema deve solicitar os campos Título da Obra e Autor. - Ao submeter dados válidos, o sistema deve persistir a obra e retornar HTTP 201 com o ID gerado. - A obra adicionada deve aparecer imediatamente na lista da tela sem recarregamento da página. - Campos obrigatórios em branco devem ser validados antes do envio à API.
Cenários BDD	Cenário 1: Adição de obra com sucesso Dado que o usuário está autenticado na tela de Acervo Quando preenche Título 'Ética a Nicômaco' e Autor 'Aristóteles' E clica em Adicionar ao Acervo Então o sistema retorna HTTP 201 e salva a obra no banco E a obra aparece na lista #listaLivros da tela Cenário 2: Tentativa de adição sem título Dado que o usuário está na tela de Acervo Quando deixa o campo Título em branco e clica em Adicionar Então o sistema exibe 'O título da obra é obrigatório' E não envia requisição POST à API

US03 — Listar Obras do Acervo

História	Como estudante autenticado, quero visualizar todas as obras cadastradas no meu acervo, para que eu possa consultá-las e gerenciá-las.
Critérios de Aceitação	- A listagem deve ser carregada automaticamente ao acessar a tela de Acervo. - Cada item deve exibir título (em negrito), autor e botões Editar e Excluir. - Se o acervo estiver vazio, o sistema deve exibir mensagem informativa. - A lista deve ser atualizada sem recarregamento após adição ou exclusão de obra.
Cenários BDD	Cenário 1: Listagem com obras cadastradas Dado que o usuário está autenticado e há obras no acervo Quando a tela de Acervo é carregada Então o sistema executa GET /api/livros e retorna HTTP 200 E todas as obras são exibidas em #listaLivros com título, autor, Editar e Excluir Cenário 2: Acervo vazio Dado que o usuário está autenticado e não há obras cadastradas Quando a tela de Acervo é carregada Então o sistema retorna HTTP 200 com array vazio [] E exibe mensagem 'Nenhuma obra cadastrada ainda'

US04 — Editar Obra Existente

História	Como estudante autenticado, quero editar o título ou o autor de uma obra já cadastrada, para corrigir informações incorretas ou desatualizadas.
CrITÉrios de Aceitação	- O usuário deve poder acionar a edição pelo botão Editar ao lado de cada obra. - O sistema deve pré-preencher os campos com os dados atuais da obra. - O PUT deve manter o mesmo ID da obra durante toda a operação. - O sistema deve retornar HTTP 200 após atualização bem-sucedida. - A lista deve refletir os dados atualizados imediatamente após a edição.
Cenários BDD	Cenário 1: Edição bem-sucedida de título Dado que o usuário está autenticado e visualiza a obra 'O Príncipe' de 'Maquiavel' Quando clica em Editar, altera o título para 'O Príncipe - Ed. Revisada' e confirma Então o sistema envia PUT /api/livros/{id} com o novo título E retorna HTTP 200 e atualiza o item na lista Cenário 2: Cancelamento da edição Dado que o usuário abriu o formulário de edição Quando clica em Cancelar sem alterar nenhum campo Então o sistema fecha o formulário sem enviar requisição E a obra permanece inalterada na lista

US05 — Excluir Obra do Acervo

História	Como estudante autenticado, quero excluir uma obra do meu acervo informando o seu ID, para remover registros que não são mais necessários.
CrITÉrios de Aceitação	- O usuário deve poder acionar a exclusão pelo botão Excluir ao lado de cada obra. - O sistema deve solicitar confirmação antes de efetivar a exclusão. - O sistema deve retornar HTTP 200 após exclusão bem-sucedida. - A obra deve ser removida imediatamente da lista exibida na tela.
Cenários BDD	Cenário 1: Exclusão confirmada Dado que o usuário está autenticado e visualiza a obra 'fdsfds - 4534' Quando clica em Excluir e confirma a exclusão Então o sistema envia DELETE /api/livros/{id} E retorna HTTP 200 e remove o item da lista #listaLivros Cenário 2: Exclusão cancelada Dado que o usuário clicou em Excluir em uma obra Quando cancela a confirmação Então nenhuma requisição DELETE é enviada e a obra permanece no acervo

9. Repositório e Materiais

Código-Fonte	https://github.com/CaioMurilo/projeto-archivum-testes/tree/main
---------------------	---

Vídeos (Drive)	https://drive.google.com/drive/folders/1niBPoCNpDFy5gylRXElf-t0i-5Y9ak3d
Ferramentas	Python 3.13 + Flask SQLite3 Pytest 8.3 Playwright 0.8 Node.js (frontend)
Disciplina	Teste de Software — Engenharia de Software, UCB Taguatinga, 2026/1